Fitflix Frontdesk & Backend: Edge Cases & Failure Points

Complete audit across 9 features · Plain English reference document · Total 32 Discovered Edge Cases

SYSTEM SUMMARY TABLE

#	Feature Area	Edge Cases	Status
Step 1	Authentication & Staff Sign-in	**4**	Audited & Logged
Step 2	New Sales & Invoicing	**4**	Audited & Logged
Step 3	Memberships & Renewals	**5**	Audited & Logged
Step 4	Leads & Pipeline	**4**	Audited & Logged
Step 5	Member Onboarding Timeline	**3**	Audited & Logged
Step 6	Slots & Attendance	**3**	Audited & Logged
Step 7	Doctor & Nutritionist Appointments	**2**	Audited & Logged
Step 8	Credits & Ledger	**4**	Audited & Logged
Step 9	Workout Builder (Client-side)	**3**	Audited & Logged
Step 10	Services & Therapies Catalog	**3**	Audited & Logged
Step 11	Dashboard Overview & Operational Alerts	**3**	Audited & Logged
Step 12	Admin Settings, Locations & Permissions	**2**	Audited & Logged
Step 13	Trainer Member Roster & PT Status	**2**	Audited & Logged
Step 14	Personal Training (PT) Session & Set Logging	**2**	Audited & Logged
Step 15	Workout Plan Assignment & Calendar Scheduling	**2**	Audited & Logged
Total Audited Items		**46**	**All 9 Features Complete**

Feature 1: Authentication & Staff Sign-in

EC-001 Frontdesk Staff Lands on the Wrong Screen After Sign-in

Real-life Situation: A frontdesk receptionist arrives at the gym, types their staff email and password on the login screen, and clicks "Sign in".

What Goes Wrong: Instead of taking them to the front-desk reception screen (where they can check in members, take bookings, or register walk-ins), the system opens up the personal workout-routine builder. They have to manually click through the menu to find their reception desk.

Why It Happens: The sign-in system has specific destination addresses saved for trainers and dietitians, but it never saved a destination address for regular frontdesk staff. When it doesn't know where to send someone, it defaults to the workout builder screen.

Safe Suggestion: Teach the sign-in redirect system that when a "frontdesk" or "staff" user logs in, their starting home screen should automatically be the Frontdesk Operations Overview (`/admin/overview`) or Bookings list.

EC-002 Old Login Data Stays Stuck in the Browser

Real-life Situation: A staff member gets logged out after their session times out and ends up back on the login page. Another staff member steps up to log in on the same computer.

What Goes Wrong: The previous staff member's digital pass is still lingering in the browser's hidden memory. In rare cases, this can cause the system to briefly mix up who is currently at the computer or fail to complete the new login cleanly.

Why It Happens: The automatic clean-up routine has a rule that says "if you are already on the login page, don't clear the saved memory." This was meant to avoid repeating clean-up actions, but it accidentally prevents old passes from being wiped out.

Safe Suggestion: Whenever anyone lands on the login page or an expired pass is rejected, immediately wipe all lingering passes from the computer's temporary memory so the next login starts 100% fresh.

EC-003 Staff Suddenly Logged Out Exactly After 24 Hours

Real-life Situation: A frontdesk computer is left running across day shifts or an evening-to-morning handover. The staff member is in the middle of talking to a customer and clicks to open a customer's record.

What Goes Wrong: The screen suddenly kicks them back to the login page without warning, interrupting their customer conversation.

Why It Happens: The browser creates a temporary daily security badge that expires in exactly 24 hours. Even if the gym's main central server is still happy with the staff member's login pass, the front door security on the web server notices the 24-hour daily badge has expired and immediately blocks access.

Safe Suggestion: When the daily badge is close to expiring, check if the staff member is still actively working with a valid server pass, and silently extend the daily badge so staff aren't kicked out mid-shift.

EC-004 Passwords Stored in Plain View Inside Browser Storage

Real-life Situation: A staff member signs into the frontdesk dashboard. They work normally.

What Goes Wrong: The system saves a slightly scrambled (base64) copy of their actual typed password into the browser's internal storage box on that computer. If another person inspects the computer's browser storage, they could unscramble and read the password.

Why It Happens: The app has an older piece of code from an earlier version that tried to remember the username and password just in case the new digital pass system didn't work.

Safe Suggestion: Modern industry standard only uses digital temporary passes (tokens) and never saves the actual password in the browser storage. We can safely stop saving the password into the browser box, relying entirely on the secure digital pass.

Feature 2: New Sales & Invoicing

EC-005 "Collect Later" Button Teleports Staff to Workout Routines Instead of Invoices

Real-life Situation: A customer agrees to purchase a membership. Staff creates the invoice, but the customer says, "I left my phone in my car, I will pay in the afternoon." Staff clicks "Collect Later" to save the pending bill.

What Goes Wrong: Instead of taking staff back to their list of unpaid invoices or the sales counter, the screen suddenly teleports them to the workout exercise planner (`/dashboard`). Staff is confused and has to click back to the admin menu.

Why It Happens: The "Collect Later" button in the sales screen was hardwired with a destination of `/dashboard` instead of the invoices page (`/admin/invoices`).

Safe Suggestion: Redirect staff to `/admin/invoices` when clicking "Collect Later", so they immediately see the pending bill at the top of their unpaid queue.

EC-006 Unpaid Invoices Cannot Be Cancelled from the Invoices List

Real-life Situation: A member changed their mind, or staff accidentally created a duplicate pending bill. Later, staff goes to the main Invoices screen (`/admin/invoices`), clicks on the unpaid invoice, and opens the side panel.

What Goes Wrong: The side panel only has a "Mark as Paid" button and a "Download PDF" button. There is no button to cancel or void the invoice. The mistaken bill remains stuck under "Awaiting Payment" forever.

Why It Happens: The ability to cancel an invoice was only coded inside the initial sale wizard, but was forgotten on the general Invoices drawer.

Safe Suggestion: Add a "Cancel Invoice" button directly inside the Invoice side drawer for any invoice that is in Draft or Pending status.

EC-007 Complimentary (₹0) Memberships Force Staff to Fake a Payment Method

Real-life Situation: A manager approves a 100% discount for a VIP or promotional trial. The invoice total is ₹0. Staff clicks to record payment.

What Goes Wrong: The popup forces staff to choose between "Cash", "UPI", "Card", or "Bank Transfer". Staff is forced to falsely click "Cash" or "Card" for a ₹0 transaction, which confuses the gym's end-of-day cash drawer accounting.

Why It Happens: The central server supports recording ₹0 transactions with "None" as the payment method, but the frontdesk popup only provides cash/card/UPI buttons.

Safe Suggestion: If an invoice total is ₹0, automatically select "Complimentary (₹0)" so accounting reports stay 100% accurate.

EC-008 Membership Date Clashes Show a Blind Error Without Help

Real-life Situation: A member already has an active gym membership that runs until December 15. Staff goes to the Invoices screen to mark their renewal invoice as paid, but the renewal start date was entered as today.

What Goes Wrong: Staff clicks "Confirm Payment". A red error message pops up saying "A membership already covers this period", and it refuses to complete. Staff is NOT told which membership is clashing or when the old membership ends.

Why It Happens: While the initial sales wizard knows how to read the clash details, the standalone "Mark as Paid" popup ignores the conflict information and only displays the raw error text.

Safe Suggestion: In the payment popup, read the clash details and show a helpful prompt: "This member's current plan ends on Dec 15. Would you like to start this plan on Dec 16?" with a one-click button.

Feature 3: Memberships & Renewals

EC-009 Expired Members Mysteriously Vanish on the First Day of the Month

Real-life Situation: It's the 1st of the month (e.g. November 1st). Staff opens the "Renewals" screen (`/admin/renewals`) to call members whose plans just expired yesterday (October 31st).

What Goes Wrong: The members whose memberships expired yesterday or 2 days ago completely disappear from the Overdue list! The list shows 0 overdue members on the 1st of every month.

Why It Happens: The code has a strict calendar rule that says: "Only look at members whose end-date belongs to this exact month." Because yesterday was in last month's calendar, the system skips them completely.

Safe Suggestion: Change the overdue list so it looks back at anyone who expired in the last 30 or 60 days, regardless of which calendar month it was.

EC-010 Frontdesk Staff Blocked by Red Error When Clicking "Manual Membership"

Real-life Situation: Frontdesk staff receives approval from the gym owner to assign a complimentary athlete membership using the "+ Manual membership (no invoice)" button.

What Goes Wrong: Staff fills out the form with the required reason and clicks Save. A red error banner appears saying: *"Only admins can create memberships"*.

Why It Happens: The central server strictly limits manual membership creation to full system Administrators. Frontdesk staff (role `frontdesk`) are not allowed to make manual memberships, yet the button is displayed to them on their screen.

Safe Suggestion: Hide the "Manual membership" button for regular frontdesk staff so they aren't misled, and direct them to use the New Sale flow instead.

EC-011 Freezing (Pausing) a Membership Gives an Abrupt Error Without Warning Staff About the 30-Day Limit

Real-life Situation: A customer asks to freeze their gym membership for 45 days due to an office trip. Staff clicks the "Freeze (Pause)" button.

What Goes Wrong: The system abruptly shows an error: *"This membership has already used its 30-day freeze allowance for the term"*. Staff looks unprepared in front of the customer.

Why It Happens: The gym policy has a 30-day freeze limit per year, but the screen does not show how many freeze days the member has already used.

Safe Suggestion: Show a clear pause summary (e.g. *"Frozen: 10 / 30 days used"*) directly on the member's card so staff can check their remaining allowance before attempting to freeze.

EC-012 Renewing Recently Expired Members Loses Their Plan History

Real-life Situation: A member's plan expired 3 days ago. They come into the gym today to renew. Staff clicks the "Renew" button next to their name on the Renewals screen.

What Goes Wrong: Instead of continuing their existing plan and remembering their discount/branch settings, the system treats them as a completely brand new walk-in and resets the form to blank.

Why It Happens: The renewal helper rule says: "Only consider a membership renewable if its end date is in the future." Once a plan expires by even 1 day, the system fails to recognize it as a renewal.

Safe Suggestion: Allow any membership that expired within the last 30 days to still be renewed with prefilled plan settings starting from today.

EC-013 Screen Displays "₹NaN" if a Membership Has an Unset or Unrecognized Price

Real-life Situation: An older membership imported from a spreadsheet or created without a price field is viewed on the Memberships table.

What Goes Wrong: The price column displays **"₹NaN"** instead of ₹0 or a clean dash.

Why It Happens: When the system cleans up incoming data, it checks if credits are numbers, but it forgot to check if the price is a valid number before sending it to the screen.

Safe Suggestion: If the price is missing or not a valid number, safely default it to 0 so the screen displays "₹0.00" cleanly.

Feature 4: Leads & Pipeline

EC-014 Dragging a Lead Card into "Converted" Always Fails With an Error

Real-life Situation: A sales person is viewing the Leads board (`/admin/leads`). After a successful consultation, they drag the customer's card from "Qualified" into the "Converted" column.

What Goes Wrong: The card immediately snaps back and displays a red error: *"Leads move to Converted when their invoice is paid. Use 'Sell plan' to start the sale."*

Why It Happens: The board shows a "Converted" drop-target column, but dropping cards onto it is strictly forbidden because conversions must be backed by a paid invoice.

Safe Suggestion: When a card is dropped into "Converted", instead of showing a harsh red error, open a friendly prompt: *"Ready to convert this lead? Click here to start their membership sale"* which directly launches the New Sale flow with this lead prefilled.

EC-015 System Invents Fake Email Addresses (`noemail-...@fitflix.in`) for Phone-Only Walk-ins

Real-life Situation: A walk-in customer shares their name and phone number at the desk, but doesn't have or want to share their email. Staff saves the lead.

What Goes Wrong: The frontdesk system silently invents a fake email address like `noemail-9876543210@fitflix.in`. Later, if automated receipts or welcome messages are sent, they bounce.

Why It Happens: The frontdesk code automatically fabricates a fake email if none was typed, even though the central backend server already happily accepts leads without an email.

Safe Suggestion: Leave the email empty when the customer doesn't have one, rather than inventing fake `@fitflix.in` email addresses.

EC-016 WhatsApp Follow-Up Button Fails if the Phone Number Has a Leading Zero

Real-life Situation: Staff enters a lead's phone number as `09876543210` (with a local zero). Later, staff clicks the green "WhatsApp" button on the lead card to send a quick follow-up message.

What Goes Wrong: The WhatsApp chat opens with an error saying "Phone number is invalid", and the chat refuses to open.

Why It Happens: The code strips non-digit characters, but does not strip the leading zero or add the country code (`91`) needed for international WhatsApp web links.

Safe Suggestion: Automatically strip any leading zero and ensure standard country code formatting when generating WhatsApp chat links.

EC-017 Staff Blocked from Moving Walk-in Leads Directly to "Qualified"

Real-life Situation: A customer walks into the gym in person for a direct consultation. They are already standing at the desk, so there is no need to make a "cold phone call". Staff tries to drag them directly from "New" to "Qualified".

What Goes Wrong: The board throws an error: *"Cannot skip stages. Move one step at a time."* Staff is forced to awkwardly drag to "Contacted" first, wait for the screen to refresh, and then drag to "Qualified".

Why It Happens: The board enforces an inflexible rule that every lead must go through every single step in strict single-file order.

Safe Suggestion: Allow staff to jump from "New" directly to "Qualified" if they specify that this was an in-person walk-in consultation.

Feature 5: Member Onboarding Timeline

EC-018 Timeline Pills Remain Permanently Blank for Mobile App Onboarding Steps

Real-life Situation: A new member downloads the Fitflix app on their phone, fills out their health markers, sets their health goals, and signs the digital consent form. Frontdesk staff opens their member profile on the reception computer.

What Goes Wrong: The Onboarding Timeline on Frontdesk shows 0% progress, and all pills are blank and grey. Staff falsely assumes the customer hasn't completed their onboarding.

Why It Happens: The Frontdesk timeline was hardcoded with in-gym physical tests (`ACTIVE_X_TEST`, `DNA_SAMPLE`, `VALD_TEST`). The digital steps that users actually perform in the mobile app (`HEALTH_MARKERS`, `HEALTH_GOALS`, `CONSENT`, `REPORT_UPLOAD`) are completely missing from the timeline list.

Safe Suggestion: Update the timeline component to visually display both the Member App Steps and the In-Gym Clinical Tests, so staff accurately sees what the user completed on their phone.

EC-019 Frontdesk Staff Receives Red 403 Error Trying to Check Off Appointment Steps

Real-life Situation: A member sits at the front desk and asks the receptionist to book their sports scientist or nutrition onboarding consultation. Staff tries to check off the step on the member's profile.

What Goes Wrong: The server returns a red error: *"This onboarding step is completed in the member app or by its owning workflow"*.

Why It Happens: The backend only allows frontdesk staff to mark physical tests (`ACTIVE_X_TEST`, `DNA_SAMPLE`, `VALD_TEST`). It forbids frontdesk from manually checking off appointment steps because those must be created through the actual booking scheduling engine.

Safe Suggestion: Disable manual check-boxes on appointment steps and instead provide a "Book Appointment" button that directly opens the appointment scheduling drawer.

EC-020 Onboarding Timeline Component Never Actually Mounted in User Pages

Real-life Situation: A developer created the `` component to track onboarding visually.

What Goes Wrong: Frontdesk staff never actually sees this timeline anywhere on the dashboard.

Why It Happens: The component was created as a standalone file (`components/onboarding-timeline.tsx`), but was never imported or placed into the user detail dialog or user profile page (`app/admin/users/page.tsx`).

Safe Suggestion: Mount `` inside the Member Detail dialog in `app/admin/users/page.tsx` so receptionists can actually track member progress.

Feature 6: Slots & Attendance

EC-021 Single-Branch Membership Blocked with Confusing Error at Check-In

Real-life Situation: A member with a single-club membership at Branch A visits Branch B. Staff searches for the member on the Attendance screen (`/admin/attendance`), sees they have an "Active Membership" badge, and clicks "Check In".

What Goes Wrong: The screen suddenly throws a red error: *"This membership is not valid at this branch. Upgrade to multi-club access or check in at the member's home club."* Staff was given no visual indication of the member's assigned branch before clicking.

Why It Happens: The search dropdown shows the member's name and an "Active Membership" badge, but does not display their Home Branch location.

Safe Suggestion: Display the member's Home Branch next to their name in the check-in search dropdown so staff can see branch eligibility immediately before attempting check-in.

EC-022 Therapy & Recovery Session Guests Blocked from Attendance Check-In

Real-life Situation: A customer purchases a single therapy session or ice bath / sauna credit pass (without buying an ongoing monthly gym membership). They arrive for their appointment, and staff tries to check them in on the Attendance screen.

What Goes Wrong: The check-in form explicitly blocks check-in and says: *"Check-in is only allowed for members with an active membership plan."* The guest cannot be checked in!

Why It Happens: The attendance form has a hardcoded client guard: `if (!selected.hasActiveMembership) return;`. It ignores valid single-session therapy bookings or day passes.

Safe Suggestion: Allow check-in if the customer either has an active membership OR a confirmed slot/therapy booking for today.

EC-023 Members Remain Stuck as "Currently in Gym" Forever if They Don't Manually Check Out

Real-life Situation: A member finishes their workout and walks out of the gym without scanning out or notifying the receptionist.

What Goes Wrong: The attendance dashboard continues to count them as "Currently in gym" for hours or days. The occupancy gauge remains artificially high until staff manually finds their name and clicks "Check Out".

Why It Happens: The system has no auto-checkout or max-duration assumption for open visits.

Safe Suggestion: Show an "extended stay" alert or automatically flag visits exceeding 3 hours so staff can clear lingering ghost check-ins with one click.

Feature 7: Doctor & Nutritionist Appointments

EC-024 Doctors Screen on Dashboard Returns 404 Not Found (Missing Backend Route)

Real-life Situation: Frontdesk staff clicks on the "Doctors" tab in the admin navigation menu (`/admin/doctors`) to view consulting physicians.

What Goes Wrong: The screen displays a loading spinner and then fails with: *"Failed to load doctors / 404 Not Found"*. Staff cannot view, add, or manage doctors.

Why It Happens: The frontend has a full Doctor management UI (`/admin/doctors`), but the central backend server never implemented or registered any `/doctors` API endpoints in `app.ts`.

Safe Suggestion: Either remove/hide the "Doctors" tab from the navigation menu if doctors are no longer supported, or create the missing `/doctors` router in `FITFLIX_BACKEND`.

EC-025 Dietitians Attempting to Open Another Staff Member's Client Get Silent Failure

Real-life Situation: Nutritionist Priya is covering for Nutritionist Ananya who is on leave. Priya clicks to open the diet profile of Ananya's client from the appointment calendar.

What Goes Wrong: The system rejects the request with a 403 Forbidden error (`NotYourClientError`). The screen shows an empty state or generic error, preventing Priya from helping the client during Ananya's absence.

Why It Happens: The backend strictly checks client ownership (`assignedNutritionistId === requester.id`). There is no temporary coverage delegation or admin override for cross-consultation.

Safe Suggestion: Add a "Reassign / Cover Client" option for Frontdesk Admins so coverage staff can temporarily access the client's dietary history.

Feature 8: Credits & Ledger

EC-026 Frontdesk Staff Blocked with 403 Forbidden on Credits Screen & Spot-Booking Top-Up

Real-life Situation: A receptionist logs into Frontdesk with their frontdesk staff account (`staff` / `frontdesk` role) and opens the "Credits" page (`/admin/credits`) to see a member's credit balance, or uses the "Quick Top-Up" button while booking a session in spot-booking (`/admin/spot-booking`).

What Goes Wrong: The page fails with a 403 Forbidden error. The receptionist cannot see the member's credit balance, cannot view transaction history, and cannot apply quick top-ups.

Why It Happens: The backend credit routes (`GET /credits/users/:userId/balance` and `GET /credits/users/:userId/history`) only authorize `["admin", "user"]`, and `POST /credits/users/:userId/topup` only authorizes `["admin"]`. The role check does not include `"frontdesk"`.

Safe Suggestion: Add `"frontdesk"` to the authorized roles in `FITFLIX_BACKEND/src/routes/credit.routes.ts` for user balance, history, and top-up so frontdesk receptionists can assist members at the desk.

EC-027 Credit Top-Up Causes Impossible Remaining Credits (e.g., 15 Remaining out of 10 Total)

Real-life Situation: A member had a 10-credit package and used all 10 credits (leaving 0). The gym manager gives them a goodwill bonus of 5 credits via the Top-Up tool on the active membership.

What Goes Wrong: The member's profile and mobile app now display: *"Credits: 15 / 10 remaining"*. Progress bars overflow to 150%, and members get confused thinking their plan was altered or corrupted.

Why It Happens: The backend top-up function `addCreditsToMembership` increments `creditsRemaining` by +5, but leaves `creditsIncluded` unchanged at 10.

Safe Suggestion: In `addCreditsToMembership`, either increase `creditsIncluded` along with `creditsRemaining` by the topped-up amount, or show topped-up credits as a separate "Bonus Credits" line item in the UI.

EC-028 Cancelled Bookings on Expired Memberships Trap Refunded Credits in a Dead Container

Real-life Situation: A member booked a session at the end of last month on their January plan. The plan expired on January 31st. A few days into February, the session is cancelled by the trainer.

What Goes Wrong: The system logs "1 credit refunded", but when the member or receptionist checks their available balance, the credit is nowhere to be found (available balance remains 0). The member feels cheated of their credit.

Why It Happens: The backend refund service puts the credit back into the original membership that purchased it. Because that membership has already passed its expiry date (`endDate`), the active balance filter (`buildActiveMembershipFilter`) ignores it completely, trapping the refunded credit inside an expired plan.

Safe Suggestion: When refunding a booking whose membership has already expired, issue the refund into the user's current active membership, or grant it as a fresh grace credit with an extended expiry window.

EC-029 Lack of Credit Clawback/Debit Tool Leaves Accidental Over-allocations Permanent

Real-life Situation: An admin intends to add 2 bonus credits to a member's account, but accidentally types "20" and submits the top-up.

What Goes Wrong: The receptionist or admin realizes the mistake immediately, but cannot deduct or undo the 18 extra credits from the Credits screen. The top-up tool rejects negative numbers (`amount must be greater than 0`), and there is no "Debit Credits" option.

Why It Happens: The system only has a positive `addCreditsToMembership` mechanism with zero debit or void transaction support in the admin UI.

Safe Suggestion: Introduce an Admin "Debit / Adjustment" action with mandatory reason logging so managers can reverse accidental top-up errors.

Feature 9: Workout Builder (Client-side)

EC-030 Workout Builder In-Progress Draft is Completely Wiped on Browser Refresh or Navigation

Real-life Situation: A trainer spends 20–30 minutes creating a comprehensive 5-day split workout plan with detailed sets, reps, and target weights. Before clicking "Save", they accidentally click a link in the sidebar, refresh the page, or the browser tab closes.

What Goes Wrong: The entire in-progress workout plan vanishes completely and resets to a blank untitled plan. The trainer has to re-type the whole plan from scratch.

Why It Happens: The workout store persistence configuration (`stores/workout-store.ts`) uses `partialize: (state) => ({ plans: state.plans })`. It only stores completed saved plans, completely ignoring `currentPlan`, `selectedDayIndex`, and `assignedUserIds`. Furthermore, visiting `/dashboard/workouts/create` automatically executes `resetPlan()`, immediately erasing any uncommitted draft.

Safe Suggestion: Persist `currentPlan` into a dedicated browser draft key so trainers are greeted with a helpful banner: *"You have an unsaved workout draft. Would you like to restore it?"*

EC-031 Dragging an Exercise Across Sections Snaps It Back to Its Old Section

Real-life Situation: A trainer adds an exercise (such as a dynamic stretch) to Warm Up, but later decides it belongs under the main "Workout" section. They drag the card from the Warm Up box down into the Workout box.

What Goes Wrong: The card looks like it moved during the drag animation, but the moment they release the mouse, the card snaps right back into the Warm Up box.

Why It Happens: The reordering function `reorderExercisesInDay` only updates the array index (`orderIndex`), but leaves the exercise's internal `section` property unchanged (`ex.section: 'warmup'`). Because the page categorizes cards by checking `ex.section`, the card is placed back in its original section container.

Safe Suggestion: In the drag-and-drop drop handler, detect which section container the user dropped into and update `exercise.section` to match the target section.

EC-032 Deleting the Last Day Leaves a "Dead Day" State Where Exercises Cannot Be Added

Real-life Situation: A trainer creates a plan, decides to restructure it, and deletes Day 1 so they can start fresh.

What Goes Wrong: The day list becomes empty. If the trainer now opens the exercise library and clicks "Add Exercise", nothing happens. There is no error message and no exercise is added.

Why It Happens: When the last day is deleted, the days list is empty (`length: 0`), but the selected day pointer is still pointing to slot `0`. When adding an exercise, the store checks `if (!days[0]) return;` and silently does nothing.

Safe Suggestion: When all days are deleted, either automatically create a clean default "Day 1", or disable the exercise library with a prompt: *"Please click '+ Add Day' before adding exercises."*

Feature 10: Services & Therapies Catalog

EC-033 Frontdesk Staff Blocked with 403 Forbidden on Services & Therapies Catalog

Real-life Situation: A receptionist logs into Frontdesk with their frontdesk staff account (`staff` / `frontdesk` role) and opens the "Services" or "Therapies" page (`/admin/services`, `/admin/therapies`), or tries to book an ice bath / physiotherapy session for a walk-in member via Spot-Booking (`/admin/spot-booking`).

What Goes Wrong: The service dropdowns fail to load and throw a red 403 Forbidden error: *"Forbidden: Insufficient permissions"*. The receptionist cannot view the club's available services and cannot select any service or therapy to complete the member's booking.

Why It Happens: The backend route definitions in `service.routes.ts` (`GET /services`) and `therapy.routes.ts` (`GET /therapies`) only authorize `["admin", "trainer", "user"]`. They forgot to include `"frontdesk"` in the allowed roles list.

Safe Suggestion: Add `"frontdesk"` to the authorized roles in `FITFLIX_BACKEND/src/routes/service.routes.ts` and `therapy.routes.ts` for `GET /` and `GET /:id` so receptionists can freely view the service catalog.

EC-034 Deleting a Service Instantly Corrupts All Existing Member Bookings

Real-life Situation: An admin decides to discontinue a treatment (e.g. "Ice Bath Recovery") and clicks "Delete" on the Services screen.

What Goes Wrong: The service is deleted immediately. However, 15 members who had already booked upcoming ice bath slots for later this week now see their bookings show up as generic "Booking" with blank descriptions. When members arrive at the club, frontdesk staff cannot see what treatment they booked, and clicking to reschedule or open the booking throws a null error.

Why It Happens: The backend deletion function `deleteServiceById` uses `Service.findOneAndDelete` without checking the `Booking` collection. It leaves orphaned bookings pointing to a deleted service ID, with no automated cancellation, credit refund, or warning to the admin.

Safe Suggestion: Before deleting a service, check if any pending or upcoming bookings exist. If active bookings exist, block deletion and show a helpful prompt: *"Cannot delete service with 15 active bookings. Please pause or cancel bookings first."*

EC-035 Inconsistent "Pause" Capability Between Therapies and General Services

Real-life Situation: The gym's sauna or recovery room is temporarily closed for maintenance for 2 days. Staff wants to temporarily pause new bookings without deleting the service.

What Goes Wrong: If the item is classified under "Therapies", staff can click "Pause" (`isPaused: true`). But if the item is classified under standard "Services" (such as Physiotherapy or Body Composition Analysis), there is no "Pause" button at all. Staff is forced to either delete the service (corrupting data) or manually delete individual slots one by one.

Why It Happens: The `Therapy` schema includes an `isPaused` flag, but the `Service` schema never implemented the `isPaused` field.

Safe Suggestion: Add `isPaused: boolean` to the general `Service` schema and frontend management screen so all services can be temporarily paused during maintenance.

Feature 11: Dashboard Overview & Operational Alerts

EC-036 Frontdesk Overview Displays ₹0 Revenue and Blank KPIs Due to Field Name Mismatch

Real-life Situation: A club manager opens the Frontdesk Overview dashboard (`/admin/overview`) on Monday morning to inspect club revenue, active member count, and recent sales.

What Goes Wrong: Every single card displays **₹0 Revenue**, **0 Active Members**, and the recent invoices and leads lists are completely empty. The manager panics thinking all club sales data was wiped out.

Why It Happens: The frontend parser in `lib/services/dashboard.service.ts` looks for flat fields like `raw.invoices.revenue`, `raw.invoices.paid`, and `raw.memberships.active`. However, the backend aggregation in `dashboard.controller.ts` returns grouped maps (`totalsByStatus: { PAID: 185000 }` and `byStatus: { PAID: 25 }`). Because the property names do not match, the frontend defaults all numbers to 0.

Safe Suggestion: Update `normalizeDashboardMetrics` in `dashboard.service.ts` to read revenue from `raw.invoices?.totalsByStatus?.PAID ?? raw.invoices?.revenue ?? 0` and paid count from `raw.invoices?.byStatus?.PAID ?? 0`.

EC-037 Network Disruption Crashes the Overview Screen Instead of Showing Cached or Empty State

Real-life Situation: A receptionist opens the Overview screen while the gym's internet connection experiences a brief 2-second hiccup.

What Goes Wrong: Instead of showing cached stats or a friendly *"Could not refresh metrics"* banner, the entire page crashes with a red React error screen.

Why It Happens: In `dashboard.service.ts`, the developer included a fallback object (`EMPTY_METRICS`) with a comment stating it is used when the backend is unreachable. However, they forgot to put a `try/catch` block around the network call `apiClient.get('/dashboard/metrics')`, allowing the promise rejection to propagate unhandled.

Safe Suggestion: Wrap `apiClient.get` in `getMetrics()` with a `try/catch` block that catches network errors and safely returns `EMPTY_METRICS`.

EC-038 Incomplete Dashboard Endpoint Drops Recent Invoices and Recent Leads Completely

Real-life Situation: Staff looks at the "Recent Activity" widget on the Overview dashboard to see the latest walk-ins or invoices.

What Goes Wrong: The widget is permanently blank.

Why It Happens: The frontend UI expects the dashboard endpoint to return `recentInvoices` and `recentLeads` arrays. But `dashboard.controller.ts` on the backend only calculates counts and totals; it never queries or includes the recent documents in its response payload.

Safe Suggestion: Either enhance `getDashboardMetrics` on the backend to include the top 5 recent invoices and leads, or have the frontend fetch recent activity from the existing `/invoices` and `/leads` endpoints directly.

Feature 12: Admin Settings, Locations & Permissions

EC-039 Global Video Room Limits Lacks Role Guard on Central Backend Route

Real-life Situation: A club admin configures class video streaming resolution to 720p and caps capacity at 50 participants to maintain reliable internet bandwidth across branches.

What Goes Wrong: The backend endpoint `PUT /api/v1/admin/settings/rooms` lacks role protection. If any logged-in user (even a regular gym member) sends a request to this endpoint, the global video presets and participant limits for the entire network are overwritten without permission.

Why It Happens: In `src/routes/settings.routes.ts`, `settingsRouter.put("/rooms", updateConferenceSettings)` only checks that a user is logged in (`authenticateToken`), but forgot to check if their role is Admin.

Safe Suggestion: Add `authorize(["admin"])` to `PUT /api/v1/admin/settings/rooms` in `FITFLIX_BACKEND/src/routes/settings.routes.ts` so only authorized administrators can alter system-wide streaming settings.

EC-040 Frontdesk Managers Blocked with 403 Forbidden When Adjusting Alert Rules

Real-life Situation: A frontdesk branch manager wants to lower their club's "High Occupancy" alert threshold from 95% to 85% because their branch floor gets overcrowded quickly during peak hours.

What Goes Wrong: The manager tries to save the new threshold and receives a red error: *"Forbidden: Admin access required"*. They cannot adjust warnings for their own club.

Why It Happens: While the viewing route (`GET /alert-rules`) allows `frontdesk`, the update route (`PUT /alert-rules/:alertType`) strictly limits access to full system administrators (`authorize(["admin"])`).

Safe Suggestion: Allow branch managers (`frontdesk`) to update location-scoped alert rules for their own assigned club so each branch can tune alerts to their capacity.

Feature 13: Trainer Member Roster & PT Status

EC-041 Expired Members Mysteriously Haunt Trainer's Active Member Roster Indefinitely

Real-life Situation:

What Goes Wrong:

Why It Happens:

Safe Suggestion:

EC-042 Substitute Trainers Blocked from Assisting Clients When Primary Trainer Is on Leave

Real-life Situation:

What Goes Wrong:

Why It Happens:

Safe Suggestion:

Feature 14: Personal Training (PT) Session & Set Logging

EC-043 Prematurely Completing a Workout Session Permanently Locks Out Missed Sets

Real-life Situation: A trainer finishes guiding a member through a workout and taps "Complete Session". A minute later, the member says, *"Hey, we forgot to log the 3 sets of cool-down stretching!"*

What Goes Wrong: The trainer tries to add the missed sets or correct the weights on the previous exercise. The screen throws a red error: *"Session is not active / 409 Conflict"*. The trainer is permanently locked out from adding or editing any sets.

Why It Happens: The backend set-logging endpoint `logSet` strictly checks `if (session.status !== WorkoutSessionStatus.Active) return 409 Conflict`. Once marked "Completed", there is no "Reopen Session" or grace window to adjust data.

Safe Suggestion: Add a 15-minute grace window or an "Undo Complete / Reopen Session" action so trainers can record missed sets or correct weight typos.

EC-044 **Staff Logging Workout Sets Attributed as Regular Users in Database Audit Logs**

Real-life Situation: A frontdesk staff member or duty coach steps in to log a workout set on behalf of a member who doesn't have their phone with them on the gym floor.

What Goes Wrong: In the system audit trail and member history, the set is recorded as `loggedByModel: "User"` instead of `"Admin"` or staff. When managers review staff performance, these coach-assisted sessions are misattributed as self-logged by the customer.

Why It Happens: The backend helper function `actorModelForRole` only checks for `trainer` and `admin`. Any other role (such as `frontdesk`) falls back to `"User"`.

Safe Suggestion:

Feature 15: Workout Plan Assignment & Calendar Scheduling

EC-045 **Frontdesk Receptionists Blocked with 403 Forbidden When Assigning Workout Plans**

Real-life Situation: A receptionist signs up a new member at the front counter. The member asks to have the standard "Beginner 3-Day Starter" workout plan loaded onto their mobile app. The receptionist opens the plan card and clicks "Assign Member".

What Goes Wrong: The operation is rejected with a red banner: *"Forbidden: Insufficient permissions"*. The receptionist cannot assign the workout routine.

Why It Happens: The backend assignment route `POST /workout-plans/:id/assign` only authorizes `["admin", "trainer"]`. Frontdesk staff (role `frontdesk`) are excluded from attaching plans to members.

Safe Suggestion: Add `"frontdesk"` to the authorized roles in `FITFLIX_BACKEND/src/routes/workout-plan.routes.ts` for plan listing and assignment so receptionists can assign starter routines upon signup.

EC-046 **Bulk Workout Plan Assignment Hides Partial Failures from Staff**

Real-life Situation: A trainer selects 5 members from a group challenge and assigns them the "6-Week Shred" workout template starting next Monday.

What Goes Wrong: 4 members receive the routine, but 1 member already has an overlapping active plan, so their assignment fails. The system displays a green toast saying *"Plan assigned to 4 members"*, but gives no clear visual alert about which member failed or why. The 5th member comes to the gym on Monday with an empty app.

Why It Happens: The backend returns an HTTP 200 OK response with a list of successes and failures inside a `failed` array (`[{ userId, error }]`). Because the HTTP status code is 200, the UI treats it as a success and only logs the failure details to the developer console.

Safe Suggestion: When partial failures occur in bulk assignment, display a clear warning dialog listing the specific members who could not be assigned along with the exact reason (e.g. *"Pooja already has an active overlapping plan"*).